



MacroSilicon Release For
深圳市视品数字技术有限公司
Internal Use Only
2023年03月13日
MS7200
软件开发用户手册
MacroSilicon Release For
深圳市视品数字技术有限公司
Internal Use Only
2023年03月13日

目录

MS7200 SDK.....	1
移植指南	1
APPLICATION NOTE	1
1. 概述	4
2. 总体设计	4
2.1 设计理念	4
2.2 API 列表.....	5
2.2.1 软件运行相关 API	5
2.2.2 操作视频类 API	5
3. 结构体类型说明.....	5
3.1 MS_TIMING_DEFINE.....	5
3.2 MS_HDMI_DEFINE	6
3.3 MS_HDMIRX_DEFINE.....	9
4 API 使用说明.....	14
4.1 CHAR* MS7200_SDK_VERSION(VOID);	14
4.2 BOOL MS7200_CHIP_CONNECT_DETECT(UINT8 U8_CHIP_ADDR);	14
4.3 VOID MS7200_HDMIRX_INIT(VOID);	14
4.4 VOID MS7200_HDMIRX_HDCP_INIT(UINT8 *U8_RX_KSV, UINT8 *U8_RX_KEY);	15
4.5 VOID MS7200_HDMIRX_INTERRUPT_PIN_SET(HDMIRX_INT_INDEX_E E_INT_INDEX, UINT32 U32_INT_FLAG, BOOL B_SET);	15
4.6 UINT32 MS7200_HDMIRX_INTERRUPT_GET_AND_CLEAR(HDMIRX_INT_INDEX_E E_INT_INDEX, UINT32 U32_INT_FLAG, BOOL B_CLR);	15
4.7 BOOL MS7200_HDMIRX_SOURCE_CONNECT_DETECT(VOID);	15
4.8 UINT16 MS7200_HDMIRX_INPUT_CLK_GET(VOID);	16
4.9 BOOL MS7200_HDMIRX_INPUT_TIMING_GET(VIDEOTIMING_T *PTIMING);	16

4.10	VOID MS7200_HDMIRX_INPUT_INFOFRAME_GET(HDMI_CONFIG_T *PT_HDMI_RX);	16
4.11	UINT8 MS7200_HDMIRX_AUDIO_FIFO_STATUS_GET(VOID);	16
4.12	VOID MS7200_HDMIRX_HPD_CONFIG(BOOL B_CONFIG, UINT8 *U8_EDID);	16
4.13	UINT8 MS7200_HDMIRX_RXPHY_CONFIG(UINT16 *U16_TMDS_CLK);	17
4.14	VOID MS7200_HDMIRX_CORE_RESET(UINT32 U32_RESET_CTRL);	17
4.15	VOID MS7200_HDMIRX_VIDEO_CONFIG(HDMI_CONFIG_T *PT_HDMI_RX);	17
4.16	UINT16 MS7200_HDMIRX_AUDIO_CONFIG(BOOL B_CLK_CHANGE, UINT16 U16_TMDS_CLK);	17
4.17	VOID MS7200_DVOUT_INIT(DVOUT_CONFIG_T *T_DVOUT_CONFIG, BOOL B_SPDIF_OUT);	18
4.18	VOID MS7200_DVOUT_DATA_SWAP(UINT8 U8_SWAP_MODE);	18
4.19	VOID MS7200_DVOUT_PHASE_ADJUST(BOOL B_INVERT, UINT8 U8_DELAY);	18
4.20	VOID MS7200_DVOUT_VIDEO_CONFIG(BOOL B_CONFIG);	18
4.21	VOID MS7200_DVOUT_AUDIO_CONFIG(BOOL B_CONFIG);	19
5	编译选项	19
5.1	平台相关编译选项	19
6	SDK 所使用数据类型	19
7	应用指南	20
7.1	上电复位操作	20
7.2	外部 API 实现	20
7.2.1	延时函数	20
7.2.2	I2C 读写操作	20
7.3	调用举例	23
7.3.1	芯片初始化	23
7.3.2	配置芯片输入输出相关模块初始化	23
7.3.3	检测输入变化并配置	23
附录	版本历史	24

1. 概述

MS7200 芯片支持 HDMI 信号输入, 24bit 数字视频和 I2S/SPDIF 音频信号输出, 芯片引脚示意图参考 MS7200 datasheet。MS7200 的 IIC 地址可以根据 SA 引脚进行选择。当 SA 引脚上拉到电源电压或者悬空时, 地址为 0xB2。当 SA 引脚连接到 GND 时, 地址为 0x56。

MS7200 SDK LIB 是一套针对 MS7200 视频处理芯片的软件开发包。采用 ANSI C 语言编写, 软件结构采用模块化设计, 可移植性好方便进行二次开发。SDK 正式 Release 中包含以下文件:

- ms7200.h
- ms7200_comm.h
- ms7200_typedef.h

SDK 库文件

库文件命名及适用编译环境说明:

ms7200_sdklib_keil_c51_vx.x.x_202xxxxx.lib - 适用于 Keil C51;

ms7200_sdklib_keil_m0/m3/m4_vx.x.x_202xxxxx.lib - 适用于 Keil MDK, STM32 系列 MCU;

ms7200_sdklib_iar_stm8s_vx.x.x_202xxxxx.a - 适用于 IAR STM8;

ms7200_sdklib_stvd_stm8s_vx.x.x_202xxxxx.lib - 适用于 STVD STM8。

- 版本说明文件

2. 总体设计

2.1 设计理念

SDK 的接口上分为与软件运行相关和硬件操作读取相关两大类, 其中与软件相关的接口有:

- 获取 SDK 版本信息;
- 设置芯片 I2C Slave 地址, 获取芯片连接状态。

与 MS7200 硬件操作及读取相关的接口有:

- 模块初始化配置;
- 解密功能 HDCP KEY 设置;
- 获取输入的 5V 状态;
- 获取输入的时钟;
- 获取输入的时序;
- 获取输入的 Packet 信息;
- 获取中断状态;
- 配置模块状态更新
- 配置输出通道切换;
- 配置输出相位调整;
- 配置输出音视频使能。

2.2 API 列表

2.2.1 软件运行相关 API

- CHAR* ms7200_sdk_version(VOID);
- BOOL ms7200_chip_connect_detect(UINT8 u8_chip_addr);

2.2.2 操作视频类 API

- VOID ms7200_hdmirx_init(VOID);
- VOID ms7200_hdmirx_hdep_init(UINT8 *u8_rx_ksv, UINT8 *u8_rx_key);
- VOID ms7200_hdmirx_interrupt_pin_set(HDMIRX_INT_INDEX_E e_int_index, UINT32 u32_int_flag, BOOL b_set);
- UINT32 ms7200_hdmirx_interrupt_get_and_clear(HDMIRX_INT_INDEX_E e_int_index, UINT32 u32_int_flag, BOOL b_clr);
- BOOL ms7200_hdmirx_source_connect_detect(VOID);
- UINT16 ms7200_hdmirx_input_clk_get(VOID);
- BOOL ms7200_hdmirx_input_timing_get(VIDEOTIMING_T *ptTiming);
- VOID ms7200_hdmirx_input_infoframe_get(HDMI_CONFIG_T *pt_hdmi_rx);
- UINT8 ms7200_hdmirx_audio_fifo_status_get(VOID);
- VOID ms7200_hdmirx_hpd_config(BOOL b_config, UINT8 *u8_edid);
- UINT8 ms7200_hdmirx_rxphy_config(UINT16 *u16_tmds_clk);
- VOID ms7200_hdmirx_core_reset(UINT32 u32_reset_ctrl);
- VOID ms7200_hdmirx_video_config(HDMI_CONFIG_T *pt_hdmi_rx);
- UINT16 ms7200_hdmirx_audio_config(BOOL b_clk_change, UINT16 u16_tmds_clk);
- VOID ms7200_dvout_init(DVOUT_CONFIG_T *t_dvout_config, BOOL b_spdif_out);
- VOID ms7200_dvout_data_swap(UINT8 u8_swap_mode);
- VOID ms7200_dvout_phase_adjust(BOOL b_invert, UINT8 u8_delay);
- VOID ms7200_dvout_video_config(BOOL b_config);
- VOID ms7200_dvout_audio_config(BOOL b_config);

3. 结构体类型说明

3.1 MS_TIMING_DEFINE

```
typedef struct _T_MS7200_VIDEO_TIMING_
{
    UINT8          u8_polarity;
    UINT16         u16_htotal;
    UINT16         u16_vtotal;
    UINT16         u16_hactive;
```

```

UINT16      u16_vactive;
UINT16      u16_pixclk;      /*10000hz*/
UINT16      u16_vfreq;      /*0.01hz*/
UINT16      u16_hoffset;    /* h sync start to h active*/
UINT16      u16_voffset;    /* v sync start to v active*/
UINT16      u16_hsyncwidth;
UINT16      u16_vsyncwidth;
} VIDEOTIMING_T;

```

3.2 MS_HDMI_DEFINE

```

typedef enum _E_HDMI_VIDEO_CLK_REPEAT_
{

```

```

    HDMI_X1CLK      = 0x00,
    HDMI_X2CLK      = 0x01,
    HDMI_X3CLK      = 0x02,
    HDMI_X4CLK      = 0x03,
    HDMI_X5CLK      = 0x04,
    HDMI_X6CLK      = 0x05,
    HDMI_X7CLK      = 0x06,
    HDMI_X8CLK      = 0x07,
    HDMI_X9CLK      = 0x08,
    HDMI_X10CLK     = 0x09

```

```

}HDMI_CLK_RPT_E;

```

```

typedef enum _E_HDMI_VIDEO_ASPECT_RATIO_
{

```

```

    HDMI_4X3        = 0x01,
    HDMI_16X9       = 0x02

```

```

}HDMI_ASPECT_RATIO_E;

```

```

typedef enum _E_HDMI_VIDEO_SCAN_INFO_
{

```

```

    HDMI_OVERSCAN   = 0x01,    //television type
    HDMI_UNDERSCAN  = 0x02    //computer type

```

```

}HDMI_SCAN_INFO_E;

```

```

typedef enum _E_HDMI_COLOR_SPACE_
{

```

```

    HDMI_RGB        = 0x00,
    HDMI_YCBCR422   = 0x01,

```

```
HDMI_YCBCR444    = 0x02,  
HDMI_YUV420      = 0x03  
}HDMI_CS_E;
```

```
typedef enum _E_HDMI_COLOR_DEPTH_  
{  
    HDMI_COLOR_DEPTH_8BIT    = 0x00,  
    HDMI_COLOR_DEPTH_10BIT   = 0x01,  
    HDMI_COLOR_DEPTH_12BIT   = 0x02,  
    HDMI_COLOR_DEPTH_16BIT   = 0x03  
}HDMI_COLOR_DEPTH_E;
```

```
typedef enum _E_HDMI_COLORIMETRY_  
{  
    HDMI_COLORIMETRY_601     = 0x00,  
    HDMI_COLORIMETRY_709     = 0x01,  
    HDMI_COLORIMETRY_656     = 0x02,  
    HDMI_COLORIMETRY_1120    = 0x03,  
    HDMI_COLORIMETRY_SMPTE   = 0x04,  
    HDMI_COLORIMETRY_XVYCC601 = 0x05,  
    HDMI_COLORIMETRY_XVYCC709 = 0x06  
}HDMI_COLORIMETRY_E;
```

```
//HDMI vendor specific
```

```
typedef enum _E_HDMI_VIDEO_FORMAT_  
{  
    HDMI_NO_ADD_FORMAT,  
    HDMI_4Kx2K_FORMAT,  
    HDMI_3D_FORMAT  
}HDMI_VIDEO_FORMAT_E;
```

```
typedef enum _E_HDMI_4Kx2K_VIC_  
{  
    HDMI_4Kx2K_30HZ = 0x01,  
    HDMI_4Kx2K_25HZ,  
    HDMI_4Kx2K_24HZ,  
    HDMI_4Kx2K_24HZ_SMPTE  
}HDMI_4Kx2K_VIC_E;
```

```
typedef enum _E_HDMI_3D_STRUCTURE_  
{  
    HDMI_FRAME_PACKING,  
    HDMI_FIELD_ALTERNATIVE,  
    HDMI_LINE_ALTERNATIVE,
```

```
HDMI_SIDE_BY_SIDE_FULL,  
L_DEPTH,  
L_DEPTH_GRAPHICS,  
SIDE_BY_SIDE_HALF = 8  
}HDMI_3D_STRUCTURE_E;
```

```
//HDMI audio
```

```
typedef enum _E_HDMI_AUDIO_MODE_  
{
```

```
    HDMI_AUD_MODE_AUDIO_SAMPLE  = 0x00,  
    HDMI_AUD_MODE_HBR            = 0x01,  
    HDMI_AUD_MODE_DSD            = 0x02,  
    HDMI_AUD_MODE_DST            = 0x03
```

```
}HDMI_AUDIO_MODE_E;
```

```
typedef enum _E_HDMI_AUDIO_I2S_RATE_  
{
```

```
    HDMI_AUD_RATE_44K1  = 0x00,  
    HDMI_AUD_RATE_48K   = 0x02,  
    HDMI_AUD_RATE_32K   = 0x03,  
    HDMI_AUD_RATE_88K2  = 0x08,  
    HDMI_AUD_RATE_96K   = 0x0A,  
    HDMI_AUD_RATE_176K4 = 0x0C,  
    HDMI_AUD_RATE_192K  = 0x0E
```

```
}HDMI_AUDIO_RATE_E;
```

```
typedef enum _E_HDMI_AUDIO_LENGTH_  
{
```

```
    HDMI_AUD_LENGTH_16BITS  = 0x00,  
    HDMI_AUD_LENGTH_20BITS  = 0x01,  
    HDMI_AUD_LENGTH_24BITS  = 0x02
```

```
}HDMI_AUDIO_LENGTH_E;
```

```
typedef enum _E_HDMI_AUDIO_CHANNEL_  
{
```

```
    HDMI_AUD_2CH  = 0x01,  
    HDMI_AUD_3CH  = 0x02,  
    HDMI_AUD_4CH  = 0x03,  
    HDMI_AUD_5CH  = 0x04,  
    HDMI_AUD_6CH  = 0x05,  
    HDMI_AUD_7CH  = 0x06,  
    HDMI_AUD_8CH  = 0x07
```

```
}HDMI_AUDIO_CHN_E;
```

```
typedef struct _T_HDMI_CONFIG_PARA_
{
    UINT8  u8_hdmi_flag;           // FALSE = dvi out; TRUE = hdmi out
    UINT8  u8_vic;                 // reference to CEA-861 VIC
    UINT16 u16_video_clk;          // TMDS video clk, uint 10000Hz
    UINT8  u8_clk_rpt;             // enum refer to HDMI_CLK_RPT_E. X2CLK = 480i/576i, others = X1CLK
    UINT8  u8_scan_info;           // enum refer to HDMI_SCAN_INFO_E
    UINT8  u8_aspect_ratio;        // enum refer to HDMI_ASPECT_RATIO_E
    UINT8  u8_color_space;         // enum refer to HDMI_CS_E
    UINT8  u8_color_depth;         // enum refer to HDMI_COLOR_DEPTH_E
    UINT8  u8_colorimetry;         // enum refer to HDMI_COLORIMETRY_E. IT601 = 480i/576i/480p/576p,
    // others = IT709
    //
    UINT8  u8_video_format;        // enum refer to HDMI_VIDEO_FORMAT_E
    UINT8  u8_4Kx2K_vic;           // enum refer to HDMI1.4 extented resolution transmission
    UINT8  u8_3D_structure;        // enum refer to HDMI_3D_STRUCTURE_E
    //
    UINT8  u8_audio_mode;          // enum refer to HDMI_AUDIO_MODE_E
    UINT8  u8_audio_rate;          // enum refer to HDMI_AUDIO_RATE_E
    UINT8  u8_audio_bits;          // enum refer to HDMI_AUDIO_LENGTH_E
    UINT8  u8_audio_channels;       // enum refer to HDMI_AUDIO_CHN_E
    UINT8  u8_audio_speaker_locations; // 0~255, refer to CEA-861 audio infoframe, BYTE4
}HDMI_CONFIG_T;
```

3.3MS_HDMIRX_DEFINE

```
typedef enum
{
    MDT_ProgrVNegHNeg = 0x01,
    MDT_ProgrVNegHPos = 0x03,
    MDT_ProgrVPosHNeg = 0x05,
    MDT_ProgrVPosHPos = 0x07,

    MDT_InterVNegHNeg = 0x00,
    MDT_InterVNegHPos = 0x02,
    MDT_InterVPosHNeg = 0x04,
    MDT_InterVPosHPos = 0x06
}e_RxMdtPol;
```

```
typedef struct TERRORCOUNTER
```

```
{
```

```
//    BOOL b;
```

```
    UINT16 u16Count;
```

```
}ERRORCOUNTER;
```

```
//hdmi_rx_reset_ctrl
```

```
#define HDMI_RX_CTRL_MAIN    (0x00001UL)
```

```
#define HDMI_RX_CTRL_MODET (0x00002UL)
```

```
#define HDMI_RX_CTRL_HDMI   (0x00004UL)
```

```
#define HDMI_RX_CTRL_BUS    (0x00008UL)
```

```
#define HDMI_RX_CTRL_AUD    (0x00010UL)
```

```
#define HDMI_RX_CTRL_PDEC   (0x00020UL) //20180607, PACKET DECODER reset
```

```
#define HDMI_RX_CTRL_PIXEL (0x00040UL)
```

```
#define HDMI_RX_CTRL_VID    (0x00080UL)
```

```
#define HDMI_RX_CTRL_HDCP   (0x00100UL)
```

```
#define HDMI_RX_CTRL_TMDS   (0x10000UL)
```

```
typedef enum _E_HDMRX_INT_INDEX_
```

```
{
```

```
    RX_INT_INDEX_PDEC,
```

```
    RX_INT_INDEX_MDT,
```

```
    RX_INT_INDEX_HDMI,
```

```
    RX_INT_INDEX_AUDFIFO,
```

```
    RX_INT_INDEX_5V
```

```
}HDMIRX_INT_INDEX_E;
```

```
typedef enum _E_MDT_INTRRUPT_STATUS_
```

```
{
```

```
    MDT_HS_ACT_ISTS          = 0x001,
```

```
    MDT_VS_ACT_ISTS          = 0x002,
```

```
    MDT_DE_ACTIVITY_ISTS     = 0x004,
```

```
    MDT_ILACE_ISTS           = 0x008,
```

```
MDT_HTOT32_CLK_ISTS    = 0x010,  
MDT_HS_CLK_ISTS        = 0x020,  
MDT_HACT_PIX_ISTS      = 0x040,  
MDT_VTOT_CLK_ISTS      = 0x080,  
MDT_VS_CLK_ISTS        = 0x100,  
MDT_VACT_LIN_ISTS      = 0x200,  
MDT_VTOT_LIN_ISTS      = 0x400,  
MDT_VOFS_LIN_ISTS      = 0x800,  
MDT_STB_ISTS           = 0x1000,  
MDT_USTB_ISTS          = 0x2000,
```

```
//
```

```
MDT_ALL_ISTS           = 0xffff  
}MDTISTS_E;
```

```
typedef enum _E_AUDIO_FIFO_STATUS_
```

```
{  
    AFIF_TH_MIN_STS      = 0x001,  
    AFIF_TH_MAX_STS      = 0x002,  
    AFIF_THS_PASS_STS    = 0x004,  
    AFIF_UNDERFL_STS     = 0x008,  
    AFIF_OVERFL_STS      = 0x010  
}AUDIO_FIFO_STATUS_E;
```

```
//hdmirx_packet_ists
```

```
#define PD_FIFO_TH_MIN_PASS_ISTS (1UL << 0)  
#define PD_FIFO_TH_MAX_PASS_ISTS (1UL << 1)  
#define PD_FIFO_TH_START_PASS_ISTS (1UL << 2)  
#define PD_FIFO_UNDERFL_ISTS (1UL << 3)  
#define PD_FIFO_OVERFL_ISTS (1UL << 4)  
#define PD_FIFO_NEW_ENTRY_ISTS (1UL << 8)  
#define NTSCVBI_RCV_ISTS (1UL << 11)  
#define NTSCVBI_CKS_CHG_ISTS (1UL << 12)  
#define AMP_CHG_ISTS (1UL << 13)
```

```
#define AMP_RCV_ISTS (1UL << 14)
#define VSI_RCV_ISTS (1UL << 15)
#define GCP_RCV_ISTS (1UL << 16)
#define ACR_RCV_ISTS (1UL << 17)
#define AVI_RCV_ISTS (1UL << 18)
#define AIF_RCV_ISTS (1UL << 19)
#define GMD_RCV_ISTS (1UL << 20)
#define GCP_AV_MUTE_CHG_ISTS (1UL << 21)
#define ACR_CTS_CHG_ISTS (1UL << 22)
#define ACR_N_CHG_ISTS (1UL << 23)
#define AVI_CKS_CHG_ISTS (1UL << 24)
#define AIF_CKS_CHG_ISTS (1UL << 25)
#define GMD_CKS_CHG_ISTS (1UL << 26)
#define VSI_CKS_CHG_ISTS (1UL << 27)
#define DVIDET_ISTS (1UL << 28)
#define AUD_TYPE_CHG_ISTS (1UL << 29)
#define DRM_RCV_ISTS (1UL << 30)
#define DRM_CKS_CHG_ISTS (1UL << 31)
#define PDEC_ALL_ISTS (0xffffffffUL)

//hdmi_rx_hdmi_ists
#define FREQ_LOCK_ISTS (1UL << 4)
#define FREQ_UNLOCK_ISTS (1UL << 5)
#define CLK_CHANGE_ISTS (1UL << 6)
#define HS_POL_ADJ_ISTS (1UL << 8)
#define VS_POL_ADJ_ISTS (1UL << 9)
#define CTL0_CHANGE_ISTS (1UL << 10)
#define CTL1_CHANGE_ISTS (1UL << 11)
#define CTL2_CHANGE_ISTS (1UL << 12)
#define CTL3_CHANGE_ISTS (1UL << 13)
#define DCM_GCP_ZERO_FIELDS_PASS_ISTS (1UL << 14)
#define DCM_PH_DIFF_CNT_OVERFL_ISTS (1UL << 15)
#define DCM_CURRENT_MODE_CHG_ISTS (1UL << 16)
#define SCDCCFGCHANGE_ISTS (1UL << 17)
```

```
#define SCDCSCSTATUSCHANGE_ISTS      (1UL << 18)
#define SCDCTMDSCFGCHANGE_ISTS      (1UL << 19)
#define CEAVID_FULL_ISTS              (1UL << 20)
#define CEAVID_EMPTY_ISTS            (1UL << 21)
#define AKSV_RCV_ISTS                (1UL << 25)
#define VSYNC_ACT_EDGE_ISTS          (1UL << 26)
#define VS_THR_REACHED_ISTS          (1UL << 27)
#define I2CMPDONE_ISTS               (1UL << 28)
#define I2CMPNACK_ISTS               (1UL << 29)
#define I2CMP_ARBLOST_ISTS           (1UL << 30)
#define HDMI_ALL_ISTS                (0xffffffffUL)
```

```
//RX_INT_INDEX_5V
```

```
#define RX_INT_FLAG_5V_UNPLUG        (0x04)
```

```
#define RX_INT_FLAG_5V_PLUG          (0x08)
```

3.4MS_DVIN_DEFINE

```
typedef struct _T_DVOUT_CONFIG
```

```
{
```

```
    UINT8 u8_cs_mode;    //refer to DVOUT_CS_MODE_E
```

```
    UINT8 u8_bw_mode;    //refer to DVOUT_BW_MODE_E
```

```
    UINT8 u8_dr_mode;    //refer to DVOUT_DR_MODE_E
```

```
    UINT8 u8_sy_mode;    //refer to DVOUT_SY_MODE_E
```

```
}DVOUT_CONFIG_T;
```

```
typedef enum _E_DVOUT_CS_MODE_
```

```
{
```

```
    DVOUT_CS_MODE_RGB,
```

```
    DVOUT_CS_MODE_YUV444,
```

```
    DVOUT_CS_MODE_YUV422
```

```
}DVOUT_CS_MODE_E;
```

```
typedef enum _E_DVOUT_BW_MODE_
```

```
{
```

```
    DVOUT_BW_MODE_16_24BIT,
```

```
DVOUT_BW_MODE_8_12BIT  
}DVOUT_BW_MODE_E;
```

```
typedef enum _E_DVOUT_DR_MODE_  
{  
    DVOUT_DR_MODE_SDR,  
    DVOUT_DR_MODE_DDR  
}DVOUT_DR_MODE_E;
```

```
typedef enum _E_DVOUT_SY_MODE_  
{  
    DVOUT_SY_MODE_HVDE,  
    DVOUT_SY_MODE_C656,  
    DVOUT_SY_MODE_1120,  
    DVOUT_SY_MODE_1004  
}DVOUT_SY_MODE_E;
```

4 API 使用说明

4.1 CHAR* ms7200_sdk_version(VOID);

- * Function name: ms7200_sdk_version
- * Description: get the SDK version
- * Input parameters: None
- * Output parameters: None
- * Returned value: version string

4.2 BOOL ms7200_chip_connect_detect(UINT8 u8_chip_addr);

- * Function name: ms7200_chip_connect_detect
- * Description: get the chip connect status
- * Input parameters: u8_chip_addr: 0 = auto check chip addr, others = set i2c slave addr
- * Output parameters: None
- * Returned value: connect status, 0: disconnect 1: connect

4.3 VOID ms7200_hdmirx_init(VOID);

- * Function name: ms7200_hdmirx_init
- * Description: hdmi rx module init

- * Input parameters: None
- * Output parameters: None
- * Returned value: None

4.4 VOID ms7200_hdmirx_hdcp_init(UINT8 *u8_rx_ksv, UINT8 *u8_rx_key);

- * Function name: ms7200_hdmirx_hdcp_init
- * Description: set hdmi rx hdcp ksv & key
- * Input parameters: u8_rx_ksv: 5-byte
- * u8_rx_key 280-byte
- * Output parameters: None
- * Returned value: None

4.5 VOID ms7200_hdmirx_interrupt_pin_set(HDMIRX_INT_INDEX_E e_int_index, UINT32 u32_int_flag, BOOL b_set);

- * Function name: ms7200_hdmirx_interrupt_pin_set
- * Description: set which interrupt need send to int pin
- * Input parameters: e_int_index: sel int type
- * u32_int_flag: sel int bit
- * b_set: enable or disable int to pin
- * Output parameters: None
- * Returned value: None

4.6 UINT32 ms7200_hdmirx_interrupt_get_and_clear(HDMIRX_INT_INDEX_E e_int_index, UINT32 u32_int_flag, BOOL b_clr);

- * Function name: ms7200_hdmirx_interrupt_get_and_clear
- * Description: get and clear interrupt status
- * Input parameters: e_int_index: sel int type
- * u32_int_flag: sel int bit
- * b_clr: clear int status after read
- * Output parameters: None
- * Returned value: interrupt status

4.7 BOOL ms7200_hdmirx_source_connect_detect(VOID);

- * Function name: ms7200_hdmirx_source_connect_detect
- * Description: get hdmi rx 5v detect status

- * Input parameters: None
- * Output parameters: None
- * Returned value: connect status, 0: disconnect 1: connect

4.8 UINT16 ms7200_hdmirx_input_clk_get(VOID);

- * Function name: ms7200_hdmirx_input_clk_get
- * Description: get hdmi rx input clk freq
- * Input parameters: None
- * Output parameters: None
- * Returned value: tmds clk freq, unit: 10kHz

4.9 BOOL ms7200_hdmirx_input_timing_get(VIDEOTIMING_T *ptTiming);

- * Function name: ms7200_hdmirx_input_timing_get
- * Description: get hdmi rx input timing
- * Input parameters: None
- * Output parameters: ptTiming: contain h/v total, active etc.
- * Returned value: timing valid flag

4.10 VOID ms7200_hdmirx_input_infotrame_get(HDMI_CONFIG_T

*pt_hdmi_rx);

- * Function name: ms7200_hdmirx_input_infotrame_get
- * Description: get hdmi rx input infotrame
- * Input parameters: None
- * Output parameters: pt_hdmi_rx: contain AVI, AIF, VSI etc.
- * Returned value: None

4.11 UINT8 ms7200_hdmirx_audio_fifo_status_get(VOID);

- * Function name: ms7200_hdmirx_audio_fifo_status_get
- * Description: get hdmi rx audio fifo status
- * Input parameters: None
- * Output parameters: None
- * Returned value: audio fifo status, refer to AUDIO_FIFO_STATUS_E

4.12 VOID ms7200_hdmirx_hpd_config(BOOL b_config, UINT8 *u8_edid);

- * Function name: ms7200_hdmirx_hpd_config
- * Description: set hdmi rx hpd status & edid

- * if edid no need to update, set the pointer to null,
- * otherwise will pull down rx hpd signal before config edid.
- * Input parameters: b_config: hpd status
- * u8_edid: edid for rx, no change set the pointer to null
- * Output parameters: None
- * Returned value: None

4.13 UINT8 ms7200_hdmirx_rxphy_config(UINT16 *u16_tmds_clk);

- * Function name: ms7200_hdmirx_rxphy_config
- * Description: set hdmi rx hpd status & edid
- * if get clk over 5MHz, will config rxphy,
- * otherwise power down analog part.
- * Input parameters: None
- * Output parameters: u16_tmds_clk: clk input value while config
- * Returned value: bit7: 1 = input clk valid, 0 = input clk invalid
- * bit0: 1 = rxphy config done, 0 = rxphy config error

4.14 VOID ms7200_hdmirx_core_reset(UINT32 u32_reset_ctrl);

- * Function name: ms7200_hdmirx_core_reset
- * Description: reset submodule of hdmi rx core
- * Input parameters: u32_reset_ctrl: refer to hdmi_rx_reset_ctrl in typedef.h
- * Output parameters: None
- * Returned value: None

4.15 VOID ms7200_hdmirx_video_config(HDMI_CONFIG_T *pt_hdmi_rx);

- * Function name: ms7200_hdmirx_video_config
- * Description: config for deep color, color space, pixel repeat input
- * Input parameters: pt_hdmi_rx: refer to HDMI_CONFIG_T in typedef.h
- * Output parameters: None
- * Returned value: None

4.16 UINT16 ms7200_hdmirx_audio_config(BOOL b_clk_change, UINT16 u16_tmds_clk);

- * Function name: ms7200_hdmirx_audio_config
- * Description: config audio fifo, aupll for input change
- * Input parameters: b_clk_change: 1 = need to reconfig aupll, 0 = audio fifo reset

* u16_tmnds_clk: input tmnds clk, use to config aupll

* Output parameters: None

* Returned value: None

4.17 VOID ms7200_dvout_init(DVOUT_CONFIG_T *t_dvout_config, BOOL b_spdif_out);

* Function name: ms7200_dvout_mode_config

* Description: config dvout mode, only config when system init

* Input parameters: t_dvout_config: dvout mode select

* b_spdif_out: 1 = use spdif out

* Output parameters: None

* Returned value: None

4.18 VOID ms7200_dvout_data_swap(UINT8 u8_swap_mode);

* Function name: ms7200_dvout_data_swap

* Description: config dvout mode, only config when system init

* Input parameters: u8_swap_mode: 0 = D0~D23 swap, 1 = RB/UV swap

* Output parameters: None

* Returned value: None

4.19 VOID ms7200_dvout_phase_adjust(BOOL b_invert, UINT8 u8_delay);

* Function name: ms7200_dvout_phase_adjust

* Description: output phase adjust

* Input parameters: b_invert: output clk invert

* u8_delay: 1ns of one step, max 3ns

* Output parameters: None

* Returned value: None

4.20 VOID ms7200_dvout_video_config(BOOL b_config);

* Function name: ms7200_dvout_video_config

* Description: enable/disable video pad output

* Input parameters: b_config: 1 = enable output, 0 = disable output

* Output parameters: None

* Returned value: None

4.21 VOID ms7200_dvout_audio_config(BOOL b_config);

- * Function name: ms7200_dvout_audio_config
- * Description: enable/disable audio pad output
- * Input parameters: b_config: 1 = enable output, 0 = disable output
- * Output parameters: None
- * Returned value: None

5 编译选项

5.1 平台相关编译选项

MS7200 SDK 支持多种编译平台，以下是在不同平台编译时需要增加的预编译选项：

- _PLATFORM_ARM_ARM: 32bit ARM 编译器
- __KEIL_C__: C51 Keil 编译器
- __CSMC__: STVD + COSMIC STM8 编译器
- _IAR_: IAR STM8 编译器
- _PLATFORM_WINDOWS_: MFC WIN32 编译器

6 SDK 所使用数据类型

__CODE : const 类型

__XDATA: C51 xdata

__DATA: C51 data

__IDATA: C51 idata

__NEAR: STM8 @near

__IO: volatile

UINT8: 8bit 无符号整型

INT8: 8bit 符号整型

BOOL: 布尔型

CHAR: 字符型

UINT16: 16bit 无符号整型

INT16: 16bit 符号整型

UINT32: 32bit 无符号整型

INT32: 32bit 符号整型

7. 应用指南

7.1 上电复位操作

MS7200 需要上电复位操作后才能使用 SDK 进行编程操作，具体操作如下：

- 首先将与复位 PIN 连接的 GPIO 拉成低电平；
- 保持该 GPIO 低电平时间不低于 1ms；
- 最后将该 GPIO 电平拉高完成上电复位动作。

7.2 外部 API 实现

7.2.1 延时函数

- extern VOID mculib_delay_ms(UINT8 u8_ms);

函数名：mculib_delay_ms

参数：u8_ms，8bit 无符号整型，代表延时毫秒数。

返回参数：无。

用途：毫秒级软件延时。

注意事项：无。

- 微妙延时：extern VOID mculib_delay_us(UINT8 u8_us);

函数名：mculib_delay_us

参数：u8_us，8bit 无符号整型，代表延时微秒数。

返回参数：无。

用途：微秒级软件延时。

注意事项：无。

7.2.2 I2C 读写操作

MS7200 的 IIC 地址可以根据 SA 引脚进行选择。当 SA 引脚上拉到电源电压或者悬空时，地址为 0xB2。当 SA 引脚连接到 GND 时，地址为 0x56。

- extern UINT8 mculib_i2c_read_16bitx8bval(UINT8 u8_address, UINT16 u16_index);

函数名：mculib_i2c_read_16bitx8bval

参数：u8_address，8bit 无符号整型，代表 I2C 器件的从地址；

u16_index，16 位无符号整型，代表所要读取器件内部寄存器的地址。

返回参数：UINT8，读取寄存器的返回值。

用途：16bit 寄存器 I2C 器件的读操作。

注意事项：

(1) 如何确认 IIC 单读函数通讯正常？芯片复位之后，读取以下寄存器

```
Value = mculib_i2c_read_16bidx8bval(0x56,0x0000); // 寄存器地址 0x0000, 正常返回值: 0xA2
LOG1("ms7200 chip reg0 = ", Value);
Value = mculib_i2c_read_16bidx8bval(0x56,0x0001); // 寄存器地址 0x0001, 正常返回值: 0x20
LOG1("ms7200 chip reg1 = ", Value);
Value = mculib_i2c_read_16bidx8bval(0x56,0x0002); // 寄存器地址 0x0002, 正常返回值: 0x0A
LOG1("ms7200 chip reg2 = ", Value);
Value = mculib_i2c_read_16bidx8bval(0x56,0x1000); // 寄存器地址 0x1000, 正常返回值: 0x27
LOG1("ms7200 chip reg1000 = ", Value);
Value = mculib_i2c_read_16bidx8bval(0x56,0x1001); // 寄存器地址 0x1001, 正常返回值: 0xA6
LOG1("ms7200 chip reg1001 = ", Value);
```

- 16bit 寄存器地址写操作: extern BOOL mculib_i2c_write_16bidx8bval(UINT8 u8_address, UINT16 u16_index, UINT8 u8_value);

函数名: mculib_i2c_write_16bidx8bval

参数: u8_address, 8bit 无符号整型, 代表 I2C 器件的从地址;

u16_index, 16 位无符号整型, 代表所要写入器件内部寄存器的地址;

u8_value, 8bit 无符号整型, 代表所要写入目标寄存器的值。

返回参数: UINT8, 读取寄存器的返回值。

用途: 16bit 寄存器 I2C 器件的读操作。

注意事项:

(1) 如何确认 IIC 单写函数通讯正常? 读写以下寄存器:

```
UINT8 Value;
mculib_i2c_write_16bidx8bval(0x56,0x0003,0x5A); // 寄存器地址 0x0003, 写入值: 0x5A
Value = mculib_i2c_read_16bidx8bval(0x56,0x0003); // 寄存器地址 0x0003, 正常返回值: 0x5A
LOG1("ms7200 chip reg3 = ", Value);
mculib_i2c_write_16bidx8bval(0x56, 0x1000, 0x5A); // 寄存器地址 0x1000, 写入值: 0x5A
Value = mculib_i2c_read_16bidx8bval(0x56, 0x1000); // 寄存器地址 0x1000, 正常返回值: 0x5A
LOG1("ms7200 chip reg1000 = ", Value);
```

- 16bit 寄存器地址连续读操作: extern VOID mculib_i2c_burstread_16bidx8bval(UINT8 u8_address, UINT16 u16_index, UINT16 u16_length, UINT8 *pu8_value);

函数名: mculib_i2c_burstread_16bidx8bval

参数: u8_address, 8bit 无符号整型, 代表 I2C 器件的从地址;

u16_index, 16 位无符号整型, 代表所要读取器件内部寄存器的起始地址;

u16_length, 16 位无符号整型, 代表所要读取的长度。

返回参数：无。

用途：16bit 寄存器 I2C 器件的连续读操作。

注意事项：

(1) 如何确认 IIC 连读函数通讯正常？芯片复位之后，读写以下寄存器：

```
UINT8 ValueBurst[4];  
mcuilib_i2c_burstread_16bidx8bval(0x56,0x1000,4, ValueBurst);  
LOG1("ValueBurst [0] default Value = ", ValueBurst [0]);// 寄存器地址 0x1000, 正确返回值: 0x27  
LOG1("ValueBurst [1] default Value = ", ValueBurst [1]);// 寄存器地址 0x1001, 正确返回值: 0xA6  
LOG1("ValueBurst [2] default Value = ", ValueBurst [2]);// 寄存器地址 0x1002, 正确返回值: 0xD0  
LOG1("ValueBurst [3] default Value = ", ValueBurst [3]);// 寄存器地址 0x1003, 正确返回值: 0x64
```

- 16bit 寄存器地址连续写操作: extern VOID mcuilib_i2c_burstwrite_16bidx8bval(UINT8 u8_address, UINT16 u16_index, UINT16 u16_length, UINT8 *pu8_value);

函数名: mcuilib_i2c_burstwrite_16bidx8bval

参数: u8_address, 8bit 无符号整型, 代表 I2C 器件的从地址;

u16_index, 16 位无符号整型, 代表所要写入器件内部寄存器的起始地址;

u16_length, 16 位无符号整型, 代表所要写入的长度。

返回参数：无。

用途：16bit 寄存器 I2C 器件的连续写操作。

注意事项：

(1) 如何确认 IIC 连写函数通讯正常？读写以下寄存器：

//赋予初值

```
UINT8 ValueBurst[4];  
ValueBurst [0] = 0x5A;  
ValueBurst [1] = 0xA5;  
ValueBurst [2] = 0x69;  
ValueBurst [3] = 0x96;  
mcuilib_i2c_burstwrite_16bidx8bval(0x56,0x20B0,4, ValueBurst);  
mcuilib_i2c_burstread_16bidx8bval(0x56,0x20B0,4, ValueBurst);  
LOG1("ValueBurst [0] default Value = ", ValueBurst [0]);// 寄存器地址 0x20B0, 正确返回值: 0x5A  
LOG1("ValueBurst [1] default Value = ", ValueBurst [1]);// 寄存器地址 0x20B1, 正确返回值: 0xA5  
LOG1("ValueBurst [2] default Value = ", ValueBurst [2]);// 寄存器地址 0x20B2, 正确返回值: 0x69  
LOG1("ValueBurst [3] default Value = ", ValueBurst [3]);// 寄存器地址 0x20B3, 正确返回值: 0x96
```

7.3 调用举例

以下以 HDMI 输入, 24bit SDR 视频+I2S 信号输出的例子来举例说明如何用 SDK 搭建一个实例(详见 Sample Code):

- 芯片初始化;
- 配置芯片输入输出相关模块初始化;
- 检测输入变化并配置。

7.3.1 芯片初始化

初始化的过程, 首先是要对芯片进行上电复位; 其次可以通过调用 `API ms7200_chip_connect_detect` 完成对系统上 MS7200 芯片 I2C 连接的检查, 并设置正确连接的 I2C 从地址。

7.3.2 配置芯片输入输出相关模块初始化

配置 HDMI 初始化, 包括模块初始化, HDCP, EDID、HPD 配置。

配置 DVOOUT 初始化, 包括输出的色彩空间 (CS: RGB、YUV444、YUV422), 位宽 (BW: 16/24、8/12), 速率 (DR: SDR、DDR), Sync (BT601、BT656、BT1120), 音频格式 (I2S、SPDIF)。

7.3.3 检测输入变化并配置

HDMI_RX 支持检测输入的 5V 变化、时钟变化、Timing 变化、InfoFrame 变化, 再根据变化情况, 执行系统所需要的配置。



附录：版本历史

文档版本	撰写时间	变更内容
V1.0	2020.12.8	初版
V1.1	2020.12.28	1. 文档错误修正 2. sdklib 新增接口描述补充
V1.11	2020.01.11	1. 添加 IIC Address 2. 添加 IIC 读写函数验证方法
V1.12	2020.01.19	1. 修正 IIC 读写函数验证方法
V1.13	2021.11.19	更新模板